

Program 8: Jenkins + Ansible (CI + CD) — Complete Explanation from Zero

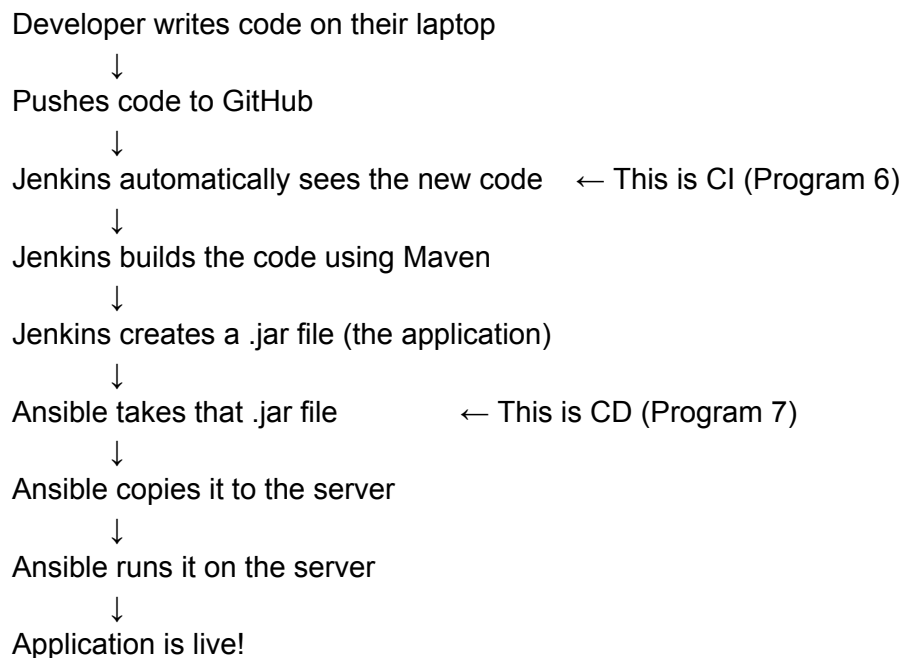
First — What is This Program About?

In Program 6 you learned Jenkins (CI — builds and tests code automatically). In Program 7 you learned Ansible (automates server tasks).

Program 8 combines both together to make a complete CI/CD pipeline.

Let me explain with a real world example:

Imagine you work at Amazon. Developers write code every day. The process should be:



CI = Jenkins builds the code CD = Ansible deploys the code

Together they form a complete automated pipeline. No human needs to do anything after the initial setup!

The Three Main Players in This Program

1. **Maven Project** — your Java application with the code
 2. **Jenkins** — watches GitHub, builds the Maven project, creates the .jar file
 3. **Ansible** — takes the .jar file Jenkins created and deploys/runs it
-

Very Important Concept — Where Files Live

This is confusing for beginners so read carefully:

You have **two environments** on your computer:

Windows side — where Jenkins runs, where your Maven project is, where the .jar file gets created

WSL/Ubuntu side — where Ansible runs

When Ansible (running in Ubuntu/WSL) wants to access a file from the Windows side, it uses a special path. In WSL, the Windows C: drive is accessible at:

`/mnt/c/`

So if Jenkins creates a .jar file at:

`C:\ProgramData\Jenkins\.jenkins\workspace\JENKINSPROJECT\target\maven-example-1.0-SNAPSHOT.jar`

Ansible in WSL accesses the same file at:

`/mnt/c/ProgramData/Jenkins/.jenkins/workspace/JENKINSPROJECT/target/maven-example-1.0-SNAPSHOT.jar`

Same file — just different way of writing the path from Windows vs Linux!

PART 1: Prepare the Maven Project

STEP 1: Make Sure You Have the Maven Project Ready

We will use the same `maven-example` project from Programs 4 and 6. It should be at:

`C:\Users\YourName\Desktop\MAVENTOGRADLEPROJECT\maven-example`

If you already did Program 6 and pushed to GitHub, you can skip to STEP 3.

If starting fresh:

```
cd desktop
mkdir MAVENTOGRADLEPROJECT
cd MAVENTOGRADLEPROJECT
mvn archetype:generate -DgroupId=com.example -DartifactId=maven-example
-DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
cd maven-example
```

STEP 2: Update pom.xml — This is the Most Important Step

This pom.xml is special compared to Program 6. We need to add `maven-jar-plugin` with the `<mainClass>` setting.

Why is this critical?

When Ansible runs the `.jar` using `java -jar filename.jar`, Java looks inside the JAR for a file called `MANIFEST.MF` which tells it which class has the `main()` method.

Without `maven-jar-plugin`, the `MANIFEST.MF` is empty and Java gives this error:

no main manifest attribute, in myapp.jar

With `maven-jar-plugin` and `<mainClass>com.example.App</mainClass>`, the `MANIFEST.MF` correctly says "run com.example.App".

Open pom.xml:

notepad pom.xml

Delete everything and paste this complete pom.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
  <modelVersion>4.0.0</modelVersion>
```

```
  <groupId>com.example</groupId>
  <artifactId>maven-example</artifactId>
  <version>1.0-SNAPSHOT</version>
```

```
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.12</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
```

```
  <build>
    <pluginManagement>
      <plugins>
```

```
        <plugin>
          <groupId>org.apache.maven.plugins</groupId>
          <artifactId>maven-compiler-plugin</artifactId>
          <version>3.0.2</version>
          <configuration>
            <source>1.8</source>
            <target>1.8</target>
          </configuration>
        </plugin>
```

```
        <plugin>
          <groupId>org.apache.maven.plugins</groupId>
          <artifactId>maven-jar-plugin</artifactId>
          <version>3.2.2</version>
          <configuration>
            <archive>
              <manifest>
                <addClasspath>true</addClasspath>
                <mainClass>com.example.App</mainClass>
```

```
        </manifest>
        </archive>
    </configuration>
</plugin>

</plugins>
</pluginManagement>
</build>

</project>
```

Press **CTRL + S** to save.

STEP 3: Make Sure App.java is Correct

Open:

notepad src\main\java\com\example\App.java

Delete everything and paste:

```
package com.example;

public class App {

    public static void main(String[] args) {
        System.out.println("Hello, Maven");
        System.out.println("This is the simple realworld example....");

        int a = 5;
        int b = 10;
        System.out.println("Sum of " + a + " and " + b
            + " is " + sum(a, b));
    }

    public static int sum(int x, int y) {
        return x + y;
    }
}
```

Press **CTRL + S** to save.

STEP 4: Test Locally First

Before involving Jenkins, always test locally to make sure Maven builds correctly.

Make sure you are in the maven-example folder:

```
cd C:\Users\YourName\Desktop\MAVENTOGRADLEPROJECT\maven-example
```

Run:

```
mvn clean compile test package
```

This is all four Maven commands in one line. You should see:

BUILD SUCCESS

Check that the JAR was created:

```
dir target
```

You should see `maven-example-1.0-SNAPSHOT.jar` in the list.

Now test running it:

```
java -cp target/maven-example-1.0-SNAPSHOT.jar com.example.App
```

Expected output:

```
Hello, Maven
This is the simple realworld example....
Sum of 5 and 10 is 15
```

If this works locally, Jenkins will also be able to build it successfully.

PART 2: Push Updated Code to GitHub

STEP 5: Push pom.xml to GitHub

Open **Git Bash** and navigate to your project:

```
cd desktop  
cd MAVENTOGRADEPROJECT  
cd maven-example
```

Now push the updated pom.xml:

```
git add pom.xml  
  
git commit -m "Add main-class to manifest"  
  
git push origin main
```

It will ask for GitHub credentials. Enter your username and Personal Access Token.

After pushing, go to <https://github.com/yourusername/SAMPLEJENKINSPROJECT> in your browser and verify the pom.xml is updated there.

Why push to GitHub? Because Jenkins pulls code FROM GitHub. Jenkins does not know about files on your local computer directly. Everything must go through GitHub.

PART 3: Set Up Jenkins

STEP 6: Open Jenkins and Configure the Job

Open browser and go to <http://localhost:8080> and log in.

If you already created **JENKINSPROJECT** in Program 6, you can use it. Just update it.

If creating fresh:

1. Click **New Item**
2. Name: **JENKINSPROJECT**

3. Select **Freestyle project**
 4. Click **OK**
-

STEP 7: Configure Source Code Management

In the job configuration page:

1. Click **Source Code Management**
2. Select **Git**
3. Repository URL:

<https://github.com/yourusername/SAMPLEJENKINSPROJECT.git>

4. Credentials: select your GitHub credentials
 5. Branch: `*/main`
-

STEP 8: Configure Build Triggers

1. Check **Poll SCM**
2. Schedule:

H/5 * * * *

This makes Jenkins check GitHub every 5 minutes for new code.

STEP 9: Configure Build Steps

This is slightly different from Program 6. We need TWO build steps:

Build Step 1 — Maven build:

1. Click **Add build step**
2. Select **Invoke top-level Maven targets**
3. Maven Version: `maven`
4. Goals:

clean compile test package

Build Step 2 — Run the JAR:

1. Click **Add build step** again
2. Select **Execute Windows batch command**
3. In the Command box type:

```
java -cp target/maven-example-1.0-SNAPSHOT.jar com.example.App
```

Click **Save**.

STEP 10: Run Jenkins Build

1. Click **Build Now**
2. Wait for build to complete
3. Click on build number → **Console Output**

Scroll to bottom. You should see:

```
Hello, Maven
This is the simple realworld example....
Sum of 5 and 10 is 15
Finished: SUCCESS
```

Jenkins part is now complete!

The JAR file is now sitting at:

```
C:\ProgramData\Jenkins\.jenkins\workspace\JENKINSPROJECT\target\maven-example-1.0-SNAPSHOT.jar
```

This is the file Ansible will pick up in the next part.

PART 4: Set Up Ansible in WSL

STEP 11: Open WSL Ubuntu Terminal

Click Start, search for **Ubuntu** and open it. Or open Command Prompt and type:

wsl

You will see the Ubuntu prompt:

itishree@DESKTOP-XXXXX:~\$

STEP 12: Install Ansible (if not already installed from Program 7)

If you already did Program 7, Ansible is installed. Verify:

```
ansible --version
```

If you see a version number, skip to STEP 13.

If not installed:

```
sudo apt update  
sudo apt install ansible -y
```

STEP 13: Install Java in WSL

Ansible needs Java installed in WSL to run the JAR file.

```
sudo apt install default-jdk -y
```

This installs the Java Development Kit. It takes a few minutes.

After installing, verify Java is working:

```
java --version
```

You should see:

```
openjdk 21.0.7 2025-04-15
```

STEP 14: Create Ansible Working Folder

```
mkdir ansible-deploy
```

```
cd ansible-deploy
```

Your prompt shows:

```
itishree@DESKTOP-XXXXX:~/ansible-deploy$
```

STEP 15: Create the Inventory File

```
nano inventory.ini
```

Type this:

```
[local]  
localhost ansible_connection=local
```

Save with **CTRL + O**, Enter, **CTRL + X**.

This is same as Program 7 — we are managing our local machine.

STEP 16: Find the Correct Path to the JAR File

This is very important. You need to find exactly where Jenkins saved the JAR file.

The JAR file on Windows is at:

```
C:\ProgramData\Jenkins\.jenkins\workspace\JENKINSPROJECT\target\maven-example-1.0-SNAPSHOT.jar
```

In WSL, the same path becomes:

```
/mnt/c/ProgramData/Jenkins/.jenkins/workspace/JENKINSPROJECT/target/maven-example-1.0-SNAPSHOT.jar
```

How to convert Windows path to WSL path:

1. Replace **C:** with **/mnt/c/**

2. Replace all \ (backslash) with / (forward slash)

Let's verify the file exists from WSL. Type:

```
ls /mnt/c/ProgramData/Jenkins/.jenkins/workspace/JENKINSPROJECT/target/
```

You should see the JAR file listed:

```
maven-example-1.0-SNAPSHOT.jar
```

If you see it, the path is correct. If not, check your Jenkins job name — the folder name matches the Jenkins job name exactly.

STEP 17: Create the Ansible Playbook (deploy.yml)

This is the most important file. It tells Ansible what to do.

```
nano deploy.yml
```

Type this content carefully. Remember — YAML is sensitive to spaces, use exactly 2 spaces for indentation, never Tab:

```
---
- name: Deploy JAR from Jenkins using Ansible
  hosts: local
  become: true
  tasks:

    - name: Copy JAR from Jenkins workspace
      copy:
        src:
/mnt/c/ProgramData/Jenkins/.jenkins/workspace/JENKINSPROJECT/target/maven-example-1.0
-SNAPSHOT.jar
        dest: /home/itishree/ansible-deploy/maven-example-1.0-SNAPSHOT.jar
        mode: '0755'

    - name: Run JAR in Background
      shell: nohup java -jar /home/itishree/ansible-deploy/maven-example-1.0-SNAPSHOT.jar >
app.log 2>&1 &
```

Replace **itishree** with your actual Ubuntu username.

Save with **CTRL + O**, Enter, **CTRL + X**.

Understanding deploy.yml Line by Line

- name: Deploy JAR from Jenkins using Ansible

Description of what this playbook does.

hosts: local

Run on machines in the **local** group from inventory.ini.

become: true

Run with sudo privileges.

- name: Copy JAR from Jenkins workspace

copy:

src: /mnt/c/ProgramData/.../maven-example-1.0-SNAPSHOT.jar

dest: /home/itishree/ansible-deploy/maven-example-1.0-SNAPSHOT.jar

mode: '0755'

Task 1 — Copy the JAR

- **copy** module copies a file from one location to another
- **src** — where the JAR is right now (Jenkins workspace on Windows side, accessed via /mnt/c/)
- **dest** — where to copy it to (inside our ansible-deploy folder in WSL)
- **mode: '0755'** — file permissions. 7 means owner can read+write+execute, 5 means others can read+execute

- name: Run JAR in Background

shell: nohup java -jar /home/itishree/ansible-deploy/maven-example-1.0-SNAPSHOT.jar > app.log 2>&1 &

Task 2 — Run the JAR

- `shell` module runs a shell command
 - `nohup` — stands for "no hang up". Keeps the process running even if terminal closes
 - `java -jar` — runs the JAR file (this works because we added `mainClass` to `pom.xml`)
 - `> app.log` — redirect normal output to `app.log` file
 - `2>&1` — redirect error output to the same `app.log` file
 - `&` — run in background (don't wait for it to finish)
-

STEP 18: Verify All Files Are Created

ls

You should see:

deploy.yml inventory.ini

STEP 19: Run the Ansible Playbook

`ansible-playbook -i inventory.ini deploy.yml --ask-become-pass`

What does `--ask-become-pass` mean? Since `become: true` is in our playbook (meaning use `sudo`), Ansible needs your `sudo` password. `--ask-become-pass` tells Ansible to ask for it.

It will show:

BECOME password:

Type your Ubuntu password and press Enter.

You will see output like this:

PLAY [Deploy JAR from Jenkins using Ansible] *****

TASK [Gathering Facts] *****

ok: [localhost]

TASK [Copy JAR from Jenkins workspace] *****

ok: [localhost]

TASK [Run JAR in Background] *****

changed: [localhost]

PLAY RECAP *****

localhost : ok=3 changed=1 unreachable=0 failed=0

Understanding PLAY RECAP:

- **ok=3** — 3 tasks ran (including Gathering Facts)
 - **changed=1** — 1 task made a change (running the JAR is new)
 - **unreachable=0** — no connection problems
 - **failed=0** — no failures. **This must be 0 for success!**
-

STEP 20: Check the Output

The JAR ran in background and its output went to **app.log**. Let's read it:

```
cat app.log
```

cat command — reads and displays a file's content.

Expected output:

Hello, Maven

This is the simple realworld example....

Sum of 5 and 10 is 15

This is the same output we got from Maven and from Jenkins!

This proves the complete CI/CD pipeline is working:

- Jenkins built the JAR ✓
 - Ansible deployed and ran the JAR ✓
 - Output is correct ✓
-

STEP 21: Run Playbook Again Without Password

You can also run without **--ask-become-pass** if the playbook does not need sudo OR if you configure sudoers. Try:

```
ansible-playbook -i inventory.ini deploy.yml
```

Testing the Complete Automated Pipeline

Now let's test that everything works end to end automatically:

Step A: Make a small change to App.java on your Windows computer. For example add:

```
System.out.println("Pipeline working!");
```

Step B: Push the change to GitHub:

```
git add .
git commit -m "test pipeline"
git push origin main
```

Step C: Wait 5 minutes for Jenkins Poll SCM to detect the change.

Step D: Jenkins automatically builds the new code and creates a new JAR.

Step E: Run Ansible playbook again:

```
ansible-playbook -i inventory.ini deploy.yml --ask-become-pass
```

Step F: Check app.log:

```
cat app.log
```

You should now see the new output including "Pipeline working!".

This is a complete CI/CD pipeline working automatically!

Complete Summary of All Commands

— PART 1: PREPARE MAVEN PROJECT (Windows CMD) —

```
cd desktop\MAVENTOGRADLEPROJECT\maven-example
notepad pom.xml      ← paste updated pom.xml with maven-jar-plugin
```



```
notepad src\main\java\com\example\App.java ← paste App.java code
mvn clean compile test package
java -cp target/maven-example-1.0-SNAPSHOT.jar com.example.App
→ verify output is correct locally
```

— PART 2: PUSH TO GITHUB (Git Bash) —

```
cd desktop\MAVENTOGRADLEPROJECT\maven-example
git add pom.xml
git commit -m "Add main-class to manifest"
git push origin main
```

— PART 3: JENKINS (browser: http://localhost:8080) —

```
New Item → JENKINSPROJECT → Freestyle project → OK
Source Code Management → Git → paste GitHub URL → credentials → branch: */main
Triggers → Poll SCM → H/5 * * * *
Build Steps → Invoke Maven → Goals: clean compile test package
Build Steps → Windows batch → java -cp target/maven-example-1.0-SNAPSHOT.jar
com.example.App
Save
Build Now → Console Output → verify SUCCESS
```

— PART 4: ANSIBLE (WSL Ubuntu terminal) —

```
sudo apt update
sudo apt install ansible -y
sudo apt install default-jdk -y
java --version
```

```
mkdir ansible-deploy
cd ansible-deploy
```

```
nano inventory.ini
→ [local]
→ localhost ansible_connection=local
→ CTRL+O, Enter, CTRL+X
```

```
ls /mnt/c/ProgramData/Jenkins/.jenkins/workspace/JENKINSPROJECT/target/
→ verify JAR file is there
```

```
nano deploy.yml
→ paste playbook content
→ CTRL+O, Enter, CTRL+X
```

```
ansible-playbook -i inventory.ini deploy.yml --ask-become-pass
→ enter password
→ verify failed=0
```

```
cat app.log
→ verify output matches expected
```

What You Write in Writeup

Aim: Set up a CI pipeline for Maven project, use Ansible to Deploy Artifacts Generated by Jenkins

Steps:

1. Create Maven project with correct pom.xml including maven-jar-plugin with mainClass
 2. Write App.java with main method and sum function
 3. Test locally with `mvn clean compile test package`
 4. Push project to GitHub using Git commands
 5. Open Jenkins, create Freestyle project called JENKINSPROJECT
 6. Configure Source Code Management with GitHub URL and credentials
 7. Set Poll SCM trigger with `H/5 * * * * schedule`
 8. Add Maven build step with goals `clean compile test package`
 9. Add Windows batch command step to run the JAR
 10. Click Build Now and verify SUCCESS in Console Output
 11. Open WSL Ubuntu terminal
 12. Install Ansible and Java using apt
 13. Create ansible-deploy folder
 14. Create inventory.ini with localhost configuration
 15. Create deploy.yml playbook with copy and shell tasks
 16. Run `ansible-playbook -i inventory.ini deploy.yml --ask-become-pass`
 17. Verify output using `cat app.log`
-

Difference Between Program 6 and Program 8

	Program 6	Program 8
What Jenkins does	Builds and runs JAR	Builds JAR only

Who runs the JAR	Jenkins (Windows batch)	Ansible (in WSL)
Ansible used?	No	Yes
Type of pipeline	CI only	CI + CD
pom.xml difference	Basic	Has maven-jar-plugin with mainClass
Run command	<code>java -cp target/...jar</code>	<code>java -jar .../...jar</code>

Viva Questions for Program 8

- **What is the difference between CI and CD?** — CI automatically builds and tests code on every push. CD automatically deploys the built application to a server
- **What is an artifact?** — The output of a build process. In this case the `.jar` file produced by Maven is the artifact
- **Why do we need maven-jar-plugin in Program 8?** — To add the main class information to the JAR's MANIFEST.MF file so that `java -jar` knows which class to run. Without it we get "no main manifest attribute" error
- **What is the difference between `java -cp` and `java -jar`?** — `java -cp` manually specifies the classpath and main class. `java -jar` runs a JAR that already has the main class in its MANIFEST.MF file
- **How does Ansible access Jenkins workspace?** — Jenkins workspace is on Windows at `C:\ProgramData\Jenkins\...` WSL can access Windows files through `/mnt/c/` path
- **What does `nohup` do?** — No hang up. Keeps the process running even after the terminal closes
- **What does `> app.log 2>&1 &` mean?** — `>app.log` redirects normal output to `app.log`, `2>&1` redirects errors to the same file, `&` runs the process in background
- **What is PLAY RECAP?** — A summary shown at the end of playbook execution showing how many tasks were ok, changed, failed or unreachable

- **What does `failed=0` mean in PLAY RECAP?** — All tasks completed successfully with no errors
 - **Why do we push to GitHub before Jenkins builds?** — Jenkins pulls code from GitHub. It does not access files directly from your computer. GitHub is the bridge between your local computer and Jenkins
 - **What is the role of Jenkins in this experiment?** — Jenkins acts as the CI tool. It monitors GitHub, pulls code, builds it using Maven and generates the .jar artifact
 - **What is the role of Ansible in this experiment?** — Ansible acts as the CD tool. It takes the .jar artifact created by Jenkins and deploys and runs it on the target machine
-

All 6 Programs — Complete One Line Summary

Program	One Line Summary
2	Create Maven project, write pom.xml and App.java, build and run using mvn commands
3	Create Gradle project with both Kotlin DSL and Groovy DSL, run using gradle commands
4	Create Maven project, migrate to Gradle using <code>gradle init</code> , verify same output from both
6	Push Maven project to GitHub, set up Jenkins freestyle job to auto-build on code push
7	Install Ansible in WSL, write playbook to install Nginx and serve custom HTML page
8	Jenkins builds Maven JAR automatically, Ansible copies and runs that JAR on local machine

You now have complete knowledge of all 6 programs from zero to execution. Is there any specific program or any specific step you are still confused about? I am happy to explain it again in even more detail!